

OUTSKILL

AI SPRINTS · SESSION NOTES

From Vibe Coding to Agentic Coding

Day 1 · AI Sprint on Windsurf and Devin

SESSION BY

Dileep Karri

Head of Generative AI Education, Outskill

Stay ahead in the AI-first world.

CONTENTS

What is in this document

PART ONE

Opening and Session Setup

Setting the Context for Day 1

PART TWO · THE CENTRAL THESIS

From Vibe Coding to Agentic Coding

The Three Steps to Agentic Coding

PART THREE · GITHUB FUNDAMENTALS

GitHub Explained Through a Pizza Restaurant

Conversations Versus Specifications

PART FOUR · THE FOUR ENGINEERING CONCEPTS

Concept One — Edge Cases

Concept Two — Authentication and Authorization

Concept Three — Rate Limits

Concept Four — Error Handling

Summary of the Four Concepts

PART FIVE · TOOLING AND TRANSITION

The History of AI-Native IDEs

Windsurf Walkthrough

PART SIX · COMMUNITY

Introducing the Outskill Illumini Forge

Community Guidelines for the Forge

CLOSING

Themes and Takeaways

Opening and Session Setup

Welcome and Framing of the AI Sprints Initiative

The session opened with Kartik welcoming participants from across time zones into the AI Sprints and Masterclasses programme. He framed the initiative as Outskill's continuing effort to create a learning environment for alumni beyond the formal cohorts, an attempt to keep delivering value once a course has ended. The AI Sprints are positioned as one part of a broader community offering — short, focused sessions that go deep into a single topic over one or two days.

For this two-day sprint, the topic was announced as Windsurf and Devin, with Day 1 dedicated to getting started with agent building and Day 2 going hands-on into building. Kartik noted that this combination — Windsurf and Devin specifically — had never been covered in any of the regular cohorts, making the sprint a chance for learners to see material outside the standard curriculum.

Ground Rules for the Session

Before handing over to the mentor, Kartik laid out a few simple but firm expectations:

Participants were asked to keep their questions written down rather than try to apply things in real time while the teaching was happening. The reasoning was that splitting attention between watching the demo and trying to follow along in their own environment usually means they miss the conceptual scaffolding the mentor is putting in place.

They were asked to be mindful in the chat — no spam, be respectful, keep it civil. The chat would be monitored, and Dileep mentioned later in the session that he had kept the chat visible only to hosts and panelists so the noise of other people's chat would not distract participants from his teaching.

Introducing the Mentor

Kartik then introduced Dileep Karri — known to the audience as KVS Dileep — as one of their favourite mentors, the Head of Gen AI Education at Outskill, and someone who has been actively helping people build AI solutions. The audience was asked to drop “boom” in the chat to call the mentor on stage, a small ritual that has become a signature of these sessions.

Dileep came on, greeted everyone, and immediately set a candid, informal tone. He apologised in advance for his “rugged look,” explained that the weekend had been tiring — half-jokingly mentioning he had almost ended up at a police station over the weekend, though everything had sorted itself out in the

end. He also mentioned wanting to take a new profile photograph, since the existing one made him look bloated. These were small humanising moments that he used to establish that the session would be conversational and unguarded, not a rehearsed corporate presentation.

Setting the Context for Day 1

What This Session Is and Is Not

Dileep then laid out, with unusual clarity, what kind of session this would be. The AI Sprints have no holds barred, in his words — meaning he does not have to constantly second-guess himself about whether the audience will understand a concept, whether something is too advanced, or whether he should hold back. In the regular cohorts there are constraints because of mixed audiences and tighter learning objectives. In the sprints, he is freer.

He levelled with the audience about the difficulty: today would be a beginner-to-intermediate level session, not advanced, but he would try to introduce some genuinely advanced concepts especially around coding. He explicitly said he was not going to do a walkthrough of Windsurf and Devin features the way a tool tutorial would. Anyone can Google those tools and find out what they do.

What he wanted to teach instead was a framework — a way of thinking about how to work with coding agents in the right manner. The tool screens were secondary; the mental model was the point.

He also told the audience to expect a whiteboard-driven session rather than a hands-on demo session. Today would be foundations; the hands-on building would come tomorrow. He asked the audience to set their expectations accordingly so that nobody felt let down because they were expecting to be coding alongside him in real time.

Permission to Drop Off

In what was perhaps the most striking framing of the opening, Dileep gave everyone explicit permission to leave. He said these are optional sessions, and if at any point someone feels the topic is too advanced, that they are getting too anxious, or that the session is not what they were looking for, they should feel free to drop off without any guilt. For the rest, his ask was simple: stay with the ride, understand what is happening, and let the session build on itself.

This kind of explicit consent-and-exit framing is not common in corporate training, and it had the effect of relaxing the audience — when nobody is being forced to stay, the people who do stay are paying attention by choice.

The Question He Opened With

Before getting into the technical material, Dileep asked the audience how many of them had shared their vibe-coded applications with friends, family, or colleagues. He followed it up with a sharper question: of those who had shared, how many had received feedback that something was broken, that this was not working, that that had gone wrong?

The answers in the chat were predictable. People had built things, shared them, and then watched them break in their friends' hands. Some had even published on the App Store and were dealing with multiple bugs. This was the audience he wanted — people who have already had the experience of “it works on my machine” colliding with the real world.

His framing of why this happens, and what to do about it, was the spine of the whole session.

The Central Thesis — From Vibe Coding to Agentic Coding

The Two-Day Arc

Dileep stated the goal of the two-day sprint plainly. Over the course of the two days, he would take participants from vibe coding to agentic coding. The previous session he had run on Claude Code covered similar territory, but this time he wanted to teach a different framework — one that would help non-coders bridge into a more structured, engineering-aware way of building.

Defining Vibe Coding

In vibe coding, the workflow is simple. You explain your requirement to an AI. The AI builds the requirement. Whenever you want to add a new feature, you have a conversation with the AI about it. The whole interaction is conversational — you describe what you want in natural language, the AI produces something, you look at it, you describe a tweak, the AI changes it, and so on.

Vibe coding lives on platforms like Bolt, Lovable, and Emergent. These platforms are specifically designed for people who do not want to think too technically — they want to express an idea and get a result. Under the hood there may be multiple agents doing serious work, but from the user's side it is a conversation.

Defining Agentic Coding

In agentic coding, the workflow is different in a way that sounds subtle but matters a lot. You do not converse — you specify. Some level of technical jargon comes into play because you are no longer waving at the AI and asking it to figure things out. You are writing a specification that tells the agent exactly what to build, often handing that specification to a swarm of agents that work together to produce the result.

A conversation is an informal chat where you are still trying to figure out what you want to build. A specification is what you produce when, in your own head, it is already absolutely clear what the agent has to do.

Where Each Style Lives

He listed the platforms where each style of work tends to happen:

Vibe coding lives on **Bolt, Lovable, and Emergent**. Agentic coding lives on **Cursor, Windsurf, Claude Code, and Codex**.

He was careful to note that the lines are blurred. You can do some vibe coding on Cursor, and you can do some agentic coding on Lovable. The platforms have been converging year on year, especially in the recent period where the gap between the two has narrowed substantially. But broadly, agentic coding is the higher level of coding — more specification, more engineering discipline, less hand-waving.

Why This Transition Matters

The reason Dileep wants people to make this transition is not philosophical. It is practical. If you want to become an AI generalist who can actually ship products, vibe coding will only take you so far. The friend who tells you the app does not work is symptomatic of the gap between conversational building and rigorous building. The transition from vibe coding to agentic coding is the transition from “it works for me” to “it works.”

The Three Steps to Agentic Coding

Dileep then laid out the three steps he sees as necessary to operate as an agentic coder, and these three steps formed the structure of everything that followed.

Step One — Understand and Be Comfortable with GitHub

The first step was being comfortable with GitHub. He said he would spend the first fifteen to twenty minutes recapping GitHub concepts in a way that would feel like a brush of memory for those who already knew it, and a real introduction for those who did not. He took an informal poll and estimated roughly a sixty-forty split between those who used GitHub day to day and those who did not.

Step Two — Learn to Write Specifications

The second step was understanding that what you give to the agent is not a conversation but a specification, and that writing good specifications is its own skill that needs to be learned.

Step Three — Agentic Building and Orchestration

The third step was agentic building and orchestration, for which Windsurf and Devin would be the tools used. He was clear that today's session would focus deeply on the planning part — the first two steps — and that the third step would be where Day 2 lived.

What the Two Days Will Build

To make the session concrete, Dileep set himself a challenge in front of the audience. Over the two days, he would attempt to build something he called Superplexity — his own elevated, advanced version of Perplexity. He acknowledged that he might not complete the build, but he expected to get it into reasonable shape. He also said he would do something most demos avoid, which is show how to work with an agentic coding setup on an existing codebase rather than always starting from scratch. From-scratch builds always look polished; the real friction of agentic coding shows up when you take an existing codebase and try to add features without breaking what is already there.

GitHub Explained Through a Pizza Restaurant

Why an Analogy

Rather than dive into GitHub vocabulary, Dileep chose to teach the entire concept through an analogy he could control end-to-end. He took an example of a restaurant — specifically a small pizza restaurant — and used it to explain every GitHub concept he wanted to cover. He told the engineers in the audience that this analogy would also give them a way to explain GitHub to their non-technical friends, which is itself a useful skill.

He picked pizza because it is universal and easy to reason about. He briefly considered sushi but admitted he did not know sushi well enough.

The Restaurant Setup

The world he set up was simple. There is a small restaurant that makes the two or three best pizzas around, along with some sides. There is a chef who runs the kitchen. The chef has a recipe book that contains the exact steps for each pizza. The chef has a team of sous chefs.

Over time the chef notices the restaurant has gone stale. The same recipes are being served day after day. He decides to use the sous chefs to evolve the existing recipes rather than start from scratch. So the sous chefs are going to take the existing pizza recipes and try to improve them.

The Naive Approach and Why It Fails

The first sous chef approaches the chef and says, “Give me the book. I will tear out the old recipe and attach my new version.” Dileep asked the audience whether this was a good idea, and the answer was obvious — it is not.

To make the point sharper, he expanded the scenario. Imagine the recipe has seven steps. Sous chef one wants to change step three. Sous chef two wants to change step five. Sous chef three wants to change step seven. They all make their changes directly on the recipe book, without informing the chef and without coordinating with each other.

Then the chef takes the recipe book and tries to make a pizza using the new seven steps. The audience was asked to predict what happens in one word. The chat filled with “chaos,” “disaster,” “mess,” “catastrophe.” That is exactly the point. When multiple people change a shared artefact directly, without any system of coordination, the result is unusable.

The Common-Sense Solution — Branches

Dileep then asked the audience for a common-sense solution, not a technical one. The intuitive answer came back: make a copy. Give each sous chef their own copy of the recipe. Let them experiment on their copy. Do not touch the original.

So the chef says, “My dear sous chefs, I know you are super enthusiastic, but you cannot kill my restaurant. I will give each of you a copy of the recipe — copy one, copy two, copy three. Make whatever changes you want on your copy.”

In GitHub terms, the original recipe is the **main** branch. Each copy is a **branch**. Each sous chef works on their branch independently. Changes on a branch do not affect the main recipe at all. This is the fundamental insight of branching — independent workstreams that do not interfere with each other.

The Pull Request and the Merge

Now imagine sous chef one experiments with step three of their copy and the pizza comes out fantastic. Better than before. Should they immediately replace step three on the chef's main recipe?

Dileep walked through the logic carefully. Yes, the improvement should eventually find its way into the main recipe — because other chefs will use that recipe in future, and the improvement should propagate. But no, sous chef one should not just do it directly.

What the sous chef should do is ask permission. “Dear chef, I made this change on step three. Please test it.” The chef then follows the modified steps, makes the pizza, tastes it, and decides whether the change is genuinely an improvement or not.

If it is, the chef merges the change into the main recipe. The main recipe is now updated. This is what GitHub calls a **pull request** — a formal ask to merge changes from a branch back into main. When accepted, the changes are **merged**.

Rejection With Feedback

Dileep extended the scenario. Sous chef two experiments with step five. They ask the chef for permission. The chef tries the new version. The pizza comes out soggy. So the chef rejects the change.

But a good chef does not just reject — they reject with feedback. “I like your thought process, but the pizza is getting too soggy at this temperature.” Now the sous chef knows what to fix. They can iterate on step five and try again.

Meanwhile, sous chef three has had their change to step seven accepted, sous chef one's step three change is in, and sous chef two is still working. The sous chefs are working in parallel, each on their own

branch, with the chef acting as the gatekeeper for what actually makes it into the main recipe. The main recipe pizza keeps getting made for customers throughout — the experimentation never disrupts the customer-facing product.

Working on copies does not affect the main recipe. The customer-facing pizza is never at risk during experimentation. Only when people are convinced that the taste has genuinely been elevated does the change get incorporated into the main recipe.

Forks — Starting Your Own Restaurant

Dileep then introduced a different concept. What if a sous chef decides they no longer want to be a sous chef? What if they want to start their own restaurant, with their own interpretation of the same pizza?

The chef is kind. He says, “Take a copy of the pizza recipe. Build your own restaurant. Innovate however you want.” The sous chef takes the recipe and leaves to start a new place.

Now the sous chef makes changes to the recipe at their own restaurant. Do those changes flow back to the original restaurant? No. The two restaurants are separate.

This is a **fork**. A fork is when someone copies an entire project to start their own independent version of it. Branches stay within the same restaurant — they will eventually try to merge back. Forks leave the restaurant entirely and live their own life.

He noted one advanced wrinkle: sometimes the original chef makes improvements to the main recipe and tells the forked sous chef, “Hey, I made some changes. Would you like to bring them into your restaurant?” The forked sous chef can choose to accept or reject. But the main flow of changes between forks is opt-in and rarely automatic.

Commits — The Hidden Backup Copies

The next concept was commit. Dileep asked how many people had used commits or had at least seen the term, and how many had seen a checkpoint feature in vibe coding tools. Many hands went up for checkpoint; fewer for commit.

To explain commit, he went back to the restaurant. The chef has just incorporated sous chef one's change to step three, and the pizza is genuinely better. The chef thinks, “This is a good moment. Let me make a note. Let me save this version.” He takes a copy of the current recipe and hides it somewhere safe. That hidden copy is a commit.

Then disaster strikes. The chef is rushed one day and accepts a change that he should not have. Maybe one of the sous chef's soggy step-five changes gets pushed through. The pizza is spoiled. What can the chef do?

He can throw away the bad version and restore the hidden copy. The disaster is reversed. Everything goes back to the last known good state.

That is what a commit does. It is a savepoint, a milestone, a restore point. The chef is being intelligent — they know mistakes will happen, so they keep secret copies along the way. If anything goes wrong, they can undo all changes and go back to a previous version.

Commits are like saved checkpoints in video games. When you cross level four and die at level five, you come back to level four, not level one. Commits work the same way — you save your progress at meaningful milestones so that if everything falls apart you do not have to start from scratch.

He also pointed out the asymmetry between merges and commits. You can merge many times — every pull request can result in a merge. But you should commit only when you are sure you have reached a real milestone, a real state worth preserving. Commits are not for routine operations; they are deliberate.

Quick Recap of the GitHub Concepts

To consolidate, Dileep walked through the concepts one more time. In the restaurant analogy:

A **branch** is when you make a copy of the recipe and work on changes that you intend to eventually fold back into the main recipe. A **fork** is when you make a copy of the recipe and start your own separate restaurant — there is no intention of merging back. A **pull request** is the formal ask from a branch back to main, saying, “I have finished my changes, please consider merging.” A **merge** happens when the pull request is accepted and the branch's changes are folded into main. A **commit** is a snapshot saved at a meaningful point so you can return to it if something later goes wrong.

He stressed why all of this matters for the rest of the session. If you want to do spec-driven development, or you want to operate agentically, you need to be very comfortable inside GitHub. The agent is going to be working on branches, making commits, raising pull requests, and you need to understand what is happening behind the scenes even if you are not running the commands yourself.

He promised that during the Superplexity build, the AI would handle most of the GitHub mechanics automatically. He was not going to walk through git commands. The reason he was explaining the

concepts was so that when the AI says “I have created a branch and committed the change,” the audience knows exactly what just happened.

There was also a brief mention of something called a **work tree**, which Dileep said is a subtle but useful concept distinct from a branch. He chose not to explain it today to avoid confusion. He noted it would be covered in Day 2.

Conversations Versus Specifications

Grandma's Recipe Versus MasterChef's Recipe

With GitHub recapped, Dileep moved to the second pillar of the session — specifications. He needed to make the difference between a conversation and a specification really concrete, and he reached for another food analogy.

A conversation is like your grandma's recipe. Your grandma never says “take one and a half teaspoons of sugar and add a quarter pinch of salt.” Your grandma says “this much” and gestures with her hand. “Then this much.” “Then take it out when it feels right.” Her recipe is encoded in her hands, her instincts, decades of practice. She is not lying to you and she is not being lazy — she just genuinely operates that way.

A specification is what you see on MasterChef. “Put one fourth of a teaspoon of baking soda. A drop of agar agar. Preheat your oven to 150 degrees. Let it heat for 12 minutes — not 10, not 15, exactly 12. Then put the cake in for 7 minutes until you get a golden glow on top. Pull it out. Drizzle a dash of honey.” Every step is precise, measurable, replicable.

The Trade-Off

Dileep was careful to point out that grandma's recipe is not bad. He took an example, but the point is not that one style is better than the other. The point is that the two styles trade off different things.

Grandma's recipes are flexible. If your measurements are slightly off, the dish still comes out decent. They tolerate variation. But they are very hard to replicate. Someone else trying to follow your grandma's recipe from a written-down version will not get the same dish. The instructions live in tacit knowledge that does not transfer.

MasterChef specifications are replicable. Anyone with the same ingredients and the same equipment can produce something close to the same dish. But they are brittle. Any one step that goes slightly off — the wrong oven temperature, an extra minute, the wrong type of flour — and the dish is ruined.

He drew the parallel to his own life. His wife bakes very precisely; he cooks conversationally. He vibes the food — feels it, smells it, tastes it as he goes. The result is that he and his wife have made a pact never to be in the kitchen together. Either he cooks, or she cooks; never both. The two styles do not mix.

Why Agentic Coding Demands Specifications

For agentic coding, you need to write MasterChef recipes. You need to specify precisely what you want — the features, the constraints, the edge cases, the error behaviour. The agent is going to execute against your specification literally, and if you have left ambiguities, the agent will fill them in with assumptions that may or may not match what you wanted.

The problem with specifications is exactly the problem MasterChef recipes have. Anything that goes off, even slightly, can wreck the result. The agent might misinterpret one ambiguity and the whole feature is broken. So the question becomes — how do you reduce errors in the specification? How do you write specs that are precise enough to drive an agent reliably?

The rest of the session was devoted to answering that question by introducing the concepts you need to think about explicitly when writing specifications.

The Four Concepts You Must Specify

Dileep framed the remaining technical content as four concepts that have to be in every serious specification. These are not the features themselves — features are obvious and everyone knows how to think about features. These are the things people forget to specify and then wonder why their app behaves badly.

He clarified that the four concepts are not in a particular order. They are simply the four things that came to mind as the most common gaps. The four are edge cases, authentication, rate limits, and error handling.

Concept One — Edge Cases

What an Edge Case Is

An edge case is a situation where your app could go wrong because of conditions you did not plan for. Dileep made this vivid with the pizza analogy. Your pizza recipe assumes a certain kind of oven. It assumes a certain temperature. It assumes certain ingredients. But what if someone tries to use the recipe with an older oven that overheats? What if someone uses a microwave instead of a conventional oven? What if someone uses a clay oven? The same recipe will produce very different results in each case.

He extended the analogy and listed common ways a pizza can go wrong. It can become soggy. It can become chalky or too hard. It can be undercooked. It can be burnt. The sauces can taste off. It can be tasteless. There are many distinct failure modes, and each one corresponds to something the recipe did not specify clearly enough.

Why Edge Cases Matter

The principle Dileep wanted to land was this: in advance, before you write your full specification, you should preemptively think about all the ways your app could go wrong. Then work backwards from those failure modes to add specifications that prevent them.

He used Starbucks coffee orders as a fun aside. Anyone who has ordered an elaborate coffee — a dash of cappuccino with oat milk with a layer of almond with foam — will know that each variation introduces new variables, new places for things to go wrong. He mentioned having FOMO once in Ireland listening to people's complex orders and trying his own version, which the barista accepted but he had no idea what he had actually ordered.

The Fine Line

The edge case is the boundary between two outcomes that are very close together. The pizza is well-cooked. The pizza is burnt. The line between these two states is very thin. The right temperature for the right amount of time produces a beautifully roasted pizza. A few minutes more and it is burnt.

Edge cases are these fine lines. They are not exotic failures; they are everyday failures that happen at the boundary of what you specified.

Mapping the Mobile App Example

Dileep then translated the analogy back to software. Imagine you have vibe-coded a mobile app. You tested it on your own iPhone. It works.

But will it work on all iPhones? An iPhone SE has a different screen. An iPhone 7 has different capabilities. An iPhone 5 has limitations. Some person somewhere will have an iPhone Original that Steve Jobs personally gifted them. On that, your app will likely fail because nothing about your design accounted for that hardware.

You cannot anticipate every edge case. That is not the goal. But you can anticipate most of them — the ones that are common, the ones that are obvious once you stop and think. If you can anticipate most of the common edge cases, you will end up with a much better application than someone who only tests on their own device.

Engineering 101

This is what Dileep meant by Engineering 101. In earlier sessions he had taught Tech 101, the fundamentals of how software works. Engineering 101 is the discipline of anticipating failure and building around it. It is the difference between a hobbyist build and a robust product.

He told the audience that the rest of the session would arm them with these engineering thinking tools so that when their vibe-coded or agent-coded applications start behaving weirdly, they would know exactly what to look for.

Surfacing Edge Cases in Planning

He briefly mentioned that during the planning phase later in the session, he would show how to deliberately surface edge cases — how to plan in a way that brings these hidden assumptions to the surface before they become bugs in the running code. The actual technique would come when he started planning Superplexity.

Concept Two — Authentication and Authorization

Why Most People Get This Wrong

When Dileep asked why authentication is important, the chat unanimously responded “security.” That is one reason, but it is not the main reason in the context he was teaching. The main reason for authentication, especially for the kinds of apps the audience is building, is personalization.

The Personalization Argument

Take a simple example — a reading app. You are on page 27 of a book. You close the app. The next time you open the app, how does the app know you were on page 27? Better yet — you open the same app on a different device, your laptop, and you are still on page 27. How does that work?

The answer is that the app is identifying you. It knows who you are across sessions and across devices, so it can pull up your reading state from wherever it stored it. That identification is what authentication provides.

Without authentication, you can still preserve state using something called local storage. Local storage is a small amount of data that the application keeps inside the user's browser or device. Dileep asked how many people had encountered local storage in their vibe-coded apps. Many had.

But local storage has a problem. If the user deletes the app, the local storage goes away. If the user's system crashes before they have authenticated, the data may not survive. If they switch devices, they have nothing on the new device. Local storage is fragile.

Authentication solves this by tying the user's state to a server-side identity. When they log in on any device, the server has their state and serves it back. The app remembers them because there is a stable identity they can re-attach to.

When to Add Authentication During the Build

Dileep gave a practical recommendation about when to add authentication during the development of an app. His rule is simple: build your app fully first, without authentication, and add authentication as one of the last features before you hand it to a user.

The reason is friction. When you are still iterating on features, having to log in every single time you want to test something is exhausting. You make a small change, you have to log back in, navigate to the feature you are testing, log back out if you are testing a different flow. It slows everything down.

Build the app first, prove the features work, and only then add the authentication layer.

He was clear that this was his own personal process and not a hard rule — others might prefer to add authentication earlier — but it was a process he wanted to share because it has saved him significant time over many builds.

Tools That Handle Authentication

He briefly mentioned the tools that make authentication easier. Replit has something called Replit Auth that handles it for you. Supabase has its own authentication system, which uses edge functions internally. He waved off the question of what an edge function is — that would be a deeper conversation, maybe for Day 2 — and just told the audience to take it for granted for now.

The Authentication versus Authorization Distinction

Dileep noticed some confusion in the chat between authentication and authorization, two words that sound similar but mean different things. He used a non-food example to clarify, partly because he sensed people were getting hungry from all the pizza talk.

He took the example of going to a cinema. **Authentication** is the ticket that gets you into the cinema. With a ticket, any ticket, you can enter the building.

But will you sit in the front row? Will you sit in the luxury circle? Will you sit in the sleeping pods? (He paused here to note that sleeping pods in cinemas are a uniquely Indian phenomenon — you can recline and watch a movie like it is a bedroom, which his international audience members might find amusing or alarming.)

Where you can sit is determined by what kind of ticket you have. That is **authorization**. Your ticket grants you specific seating permissions. The economy ticket gets you in the back; the premium ticket gets you the recliner. Both are valid, both got you into the cinema, but they grant different access.

In an app, authentication is what gets you logged in. Authorization is what determines which features you can access once you are logged in. You might have admin authorization, in which case you can access everything. You might have user authorization, in which case you can access only the user-facing features. An app needs both — authentication to know who you are, and authorization to know what you are allowed to do.

He preemptively cut off any questions about what the cinema screen or the seat number or the stage represents in the analogy. The analogy is there to explain the concept; do not push it further than the concept it is meant to illustrate.

He also noted the connection to databases. Authentication and authorization both live on top of a basic database setup that holds user records, role information, and permissions. The database is the foundation that makes both of these possible.

Concept Three — Rate Limits

What Rate Limits Are Not

Rate limits is a concept that almost everyone has heard of, but Dileep noticed people tend to think about it only in terms of cloud usage. You go over your monthly token allowance, you get rate-limited. He wanted to talk about rate limits in a different sense — the rate limits that affect the user-facing behaviour of your app.

The Guest Analogy

His framing of rate limits was domestic. Imagine guests are at your house. There is limited food because nobody anticipated how many guests would come. Your mother gives you specific instructions: “Only two pieces of pizza for you. Do not dare take the third piece — it is for the guests.” Your quantity is limited because the guests have to eat first.

In some cultures, particularly Indian culture as Dileep noted, this is a familiar dynamic. When guests come, the household members rate-limit themselves. Once the guests leave, normal service resumes, but for the duration of the visit, you are operating under constraints.

Your mother has applied a rate limit to your food. APIs work the same way.

APIs and Per-Tier Limits

When you call an API — Dileep used ElevenLabs as the example — you are typically operating at a particular tier of service. Each tier has rate limits attached. ElevenLabs at a particular tier might allow you to call their API 20 times per second.

Now imagine you launch your app and 100 people use it simultaneously. The first 20 calls per second go through. The remaining 80 calls fail. Your users get errors. They think your app is broken. You think your code is broken. The real cause is the rate limit on the API you depend on.

This happens a lot with free tiers. Free APIs have aggressive rate limits because the provider wants you to upgrade to a paid tier. But it also happens with paid APIs — being on the paid tier does not mean you have unlimited usage. There are still rate limits, and you have to know what they are.

A Real Recent Example

Dileep asked the audience if anyone could give a real example of a rate-limiting incident in the last few months. Someone in the chat mentioned Anthropic rate-limiting OpenCode (referred to as “Open Claw” in the transcript). Dileep confirmed this — Anthropic had rate-limited OpenCode at some point recently, which was a high-profile example of how rate limits play out in practice. You think you are paying for an

API, you think everything is fine, but the provider has limits in place that may bite you in unexpected ways.

Study the rate limits of every API you depend on, especially when you anticipate any kind of load or scaling.

Agent-Friendly Documentation

This led into a small but important tangent about how API documentation has evolved. Dileep observed that you no longer have to read the documentation yourself the old way. All modern documentation has become agent-friendly. It is no longer designed primarily for humans; it is designed for AI agents to consume.

He showed this in real time on the ElevenLabs documentation site. There is now a “Copy Page” button on the documentation. When you click it, you get options like “Open in Claude,” “Open in ChatGPT,” “Connect to Cursor,” “Connect to Claude Code.” The documentation site exposes itself as markdown for LLMs, so an agent landing on the page can ingest the whole thing as structured text.

This is a significant shift. The way the formatting is done, the way the structure is exposed, all of it is optimised for an agent reading it rather than a human reading it. As an agentic coder, you no longer have to manually trawl through documentation to learn an API — you can point your agent at the documentation and let the agent read it.

CLIs were mentioned as something he would cover in a future session, not in scope for today.

Concept Four — Error Handling

Edge Cases versus Error Handling

Dileep was careful to distinguish edge cases from error handling, even though they sound similar. Edge cases are where things could go wrong. Error handling is what your app does when things actually do go wrong. Together they form a complete picture of failure-aware design, but each requires its own thinking.

The Out-of-Syllabus Question

His analogy for error handling was the out-of-syllabus exam question. He asked how many people had frozen during an exam because they were hit with a question on a topic they did not realise was even part of the syllabus. The answer was, predictably, most of the audience.

That feeling — the freeze, the blank panic, the not knowing what to do — is exactly what an app experiences when it hits a situation it was not specified to handle. The user does something the developer did not anticipate. The app freezes. The screen goes blank. The user has no idea what happened.

Error handling is the discipline of telling your app, in advance, what to do when it encounters these unspecified situations.

The Guest Etiquette Analogy

He extended the guest analogy from the rate-limits section. There is etiquette at home for what to do with guests. Your mother tells you: when the guest offers you food, politely decline. Specific situation, specific response. That is straightforward.

But what about the situation she did not specify? The guest leaves their plate half-eaten and goes away. You see the leftover. Your mother never told you what to do with the leftover. You think, “Well, I can eat it.” You pop it in your mouth. Suddenly you get a withering look from your mother.

Error handled — badly. You took an unspecified situation and filled it in with your own assumption, which turned out to be the wrong assumption.

Building a Hierarchy of Error Responses

Dileep then sketched what good error handling looks like, again through the guest analogy. The mother gives a layered set of instructions:

When the guest asks if you can take the food, politely say no. That handles the basic case.

If the guest insists a second time, decline again. If they insist a third time, then reluctantly accept. But do not jump and grab it from their hand — at least act reluctantly when you take it. That handles repeated requests with social grace.

If you encounter any situation that has not been specified, ask the mother. That is the catch-all — the fallback. When you do not know what to do, escalate to someone who does.

This is exactly how error handling should work in an app. Each known error type gets a specific, well-designed response. For everything not anticipated, there is a graceful fallback — an “unspecified error” handler that lets the user know something went wrong without freezing the whole app.

The Quirky Error Screen

Dileep made the point that good error handling is visible even when you are not looking for it. He asked the audience if they had ever come across those quirky error screens that say “Sorry, this is on us, not

you” or “Looks like our hamster fell off the wheel” — friendly, humanised error pages. Those screens are error handling. The app encountered something it did not expect, but it knows how to gracefully tell the user that something is broken.

The opposite is the blank screen. Nothing loads. No message. No indication of what happened. That is unhandled error — the developer did not specify what to do, so the app does nothing useful at all.

What to Put in Your Specification

His advice to the audience was to specify error handling thoroughly in the agent specification. List the error types you can think of. For each one, specify exactly what the app should do. For everything else, specify a graceful catch-all. If the user is putting garbage into a form, validate it and respond: “What you have entered seems to be invalid, can you check and enter again?” If the API is down, tell the user the service is temporarily unavailable. If something completely unexpected happens, tell the user the app encountered a problem and apologise.

The point is that error handling should not be left to chance. It should be a core part of the specification.

Summary of the Four Concepts

Before moving on, Dileep paused to consolidate. The four concepts that have to be in every serious agentic coding specification are:

Edge cases	The places where things could go wrong because of conditions you may not have anticipated.
Error handling	What your app does when things actually go wrong.
Rate limits	The constraints on the APIs and services your app depends on.
Authentication & authorization	Who the user is and what they are allowed to do.

He acknowledged he had been deliberate about using analogies rather than precise technical definitions. The analogies are not perfect — they are explanatory devices. If anyone wants to go deeper into the precise technical definitions, they are welcome to, but the analogies should give a strong intuitive grasp of why each concept matters and how to think about it.

The History of AI-Native IDEs

The Iliad of IDEs

Dileep then transitioned to the tooling side. Before he could open Windsurf and show it to the audience, he wanted to give them the lineage — what he half-jokingly called the Iliad, the Odyssey, and the Mahabharata of IDEs. The epic backstory of how the current generation of AI coding tools came to be.

VS Code as the Common Ancestor

In the beginning, there was VS Code — Microsoft's integrated development environment. Dileep paused to define IDE for anyone unfamiliar with the term. An IDE is an Integrated Development Environment — the place where you actually do your coding. It is where text editing, file management, debugging, terminal access, and now AI agents all come together in one interface.

VS Code's significance is that it is open source. Anyone can take the VS Code codebase and build their own version on top of it. This has enabled an explosion of AI-native IDEs over the past few years.

The First Wave of Forks

The first major fork was Cursor. Cursor took VS Code, added AI capabilities deeply integrated into the editor, and built a company around it. Cursor became one of the dominant AI coding tools.

Another fork was Windsurf. Windsurf took a different path, with its own design choices and its own agent integration, but the lineage was the same — VS Code at the base, AI capabilities added on top.

Other forks have followed. Amazon has Kiro. ByteDance has Trae. There are several more. Each of them is a variation on the same core idea.

The Windsurf Story

Windsurf's history is, in Dileep's words, particularly curious. The company built its product, gained traction, and then something unusual happened. Google approached the founders. Google acquired the founding team plus a small core team — roughly eight to ten people — along with the Windsurf IP up to that point in time. Those people, along with the technology they had built, went to Google.

Google took the version of Windsurf they had acquired and used it to build something called **Antigravity**. So Antigravity is essentially a version of Windsurf as it stood when Google made the acquisition. Antigravity is Google's product now.

But here is the twist. Windsurf had a team of roughly 100 people before the acquisition. Only the founders and a small core team — eight to ten people — went to Google. The remaining seventy to eighty people stayed and continued building Windsurf. The product did not die when its founders left.

Then a second acquisition happened. A company called **Cognition** — the same company that built the autonomous coding agent **Devin** — acquired the remaining Windsurf team and the Windsurf product. The two companies merged. Windsurf became Windsurf 2.0, now with Devin integrated as part of the platform.

Windsurf forked into two. One fork became Antigravity at Google. The other fork stayed as Windsurf and became Windsurf 2.0 with Devin built in.

Why This History Matters

Dileep was not telling this story for entertainment, although it was entertaining. He was telling it because it explains why Windsurf 2.0 has its specific feel — somewhat similar to Antigravity, similar to Cursor at its base, but with its own paradigm and orchestration model. Knowing the lineage helps you anticipate what you will see when you open the tool.

He also wanted to manage expectations about Windsurf 2.0 specifically. The paradigm is slightly different from other tools, and the orchestration model is different. Some people might prefer Windsurf to Cursor or other options precisely because of these differences. That is why showing Windsurf was worth doing — not to declare it the best, but to give the audience direct exposure to a different style.

The Pizza Tasting Principle

Dileep wrapped up the lineage discussion with a principle that applied to every tool comparison question in the chat. He said he was not going to tell the audience which tool was the best. His job was to show them different types of pizza. They had to taste each one and decide which they preferred. Direct comparison requests — “Which is better, X or Y?” — were not what this session was for. The session was about giving people the variety they needed to form their own opinions.

Windsurf Walkthrough

Setting Up a New Project

Dileep then opened Windsurf live for the audience to see. He acknowledged that he would not be doing a heavy demo today, because he wanted to spend more time on the planning phase. But he wanted to give a quick tour of the interface and set up the project that would be the basis for Day 2.

He clicked “Generate a new project,” then “Select folder.” He chose to create a new folder called Superplexity. There was a brief moment where his keyboard stopped responding, which he handled with composure — explaining what he was doing while he troubleshooted the issue. Once the keyboard came back, he created the folder, opened it in Windsurf, and confirmed he trusted the authors of the workspace.

He noted that Windsurf looks very similar to Cursor and Antigravity because they all share the VS Code base. The general layout is familiar to anyone who has used any of these tools.

Cascade — Windsurf’s Agent

The Windsurf agent is called **Cascade**. Dileep pressed Command-L to start a new Cascade session — the equivalent of opening a new agentic chat with the AI. He compared this to Composer in Cursor and the agent mode in Antigravity. The underlying concept is the same across all three tools: a chat panel where you can give the agent instructions and have it operate on your codebase.

He noted that what is good about Windsurf — what he would demonstrate in more detail on Day 2 — is the way it orchestrates work. He had originally hoped to demonstrate something called **Arena**, a feature in Windsurf, but said he would do that later when more time was available.

Connecting GitHub from Inside Windsurf

For the first concrete demonstration, Dileep asked Cascade to create a simple repository in GitHub. He typed the instruction in plain English — “Make a simple repo in GitHub” — and let the agent work. He noted that all the standard agentic features — skills, MCP servers, and so on — work the same way across Windsurf, Cursor, Antigravity, and VS Code. Everyone has the same fundamental features. What differs is the orchestration and the user experience around those features.

Cascade asked for clarification: what should the repository be named? Dileep typed “SuperPlexity” and hit run. The agent created the repository, connected it to his GitHub account, and set up the project. He

briefly mentioned that for someone who is non-technical, the way to handle this is just to tell the agent: “This is my GitHub, I am a non-technical person, help me connect.” The agent will walk through the connection process step by step.

The point he was making is that the GitHub mechanics he had spent the first hour explaining are largely handled by the agent automatically. The audience does not need to memorise git commands. They need to understand what is happening conceptually so they can supervise the agent's work intelligently.

Model Selection in Windsurf

Dileep then opened the model picker in Windsurf to walk through the options. He apologised that he was not able to increase the font size further for the audience watching the screen share. He went through the model options one by one:

Adaptive	Windsurf’s auto-selection mode. Adaptive automatically balances quality and cost. The system inspects the input it is receiving and decides on the fly which model is appropriate. It costs roughly 0.5 cents per million tokens — the cheapest tier.
SWE 1.6 Fast	Windsurf’s own in-house coding model. This is analogous to Composer being Cursor’s in-house coding model. Windsurf has been building their own model for coding specifically, and this is their offering. Dileep noted it is competitive on price.
Kimi K2.6	A model available freely on Windsurf. He noted it is very similar in capability to Claude Code, which made it a strong option for people on the free tier who want serious coding capability without paying.
DeepSeek	Slightly more expensive than the cheapest options but still reasonable.
Claude Opus	At the top end of the price range, very costly per token but with correspondingly strong capability.
GPT 5.5	Higher than adaptive in cost, in the upper-middle range.

He noted that some of these models — specifically Adaptive and SWE 1.6 — are Windsurf-specific features. They do not exist in Antigravity. Antigravity does not have an adaptive mode, and it does not

have SWE 1.6 because that is Windsurf's proprietary model. This is one of the ways the two former siblings have diverged.

Claude Code via API Key

One subtle quirk of Windsurf that Dileep flagged: if you want to use Claude Code through Windsurf, you cannot just toggle it on. Windsurf does not have a native integration. Instead, they ask you to provide an API key and connect it manually. So Claude Code is available, but with an additional setup step compared to the other models that are integrated more smoothly.

How to Choose a Model

His default recommendation was to start with Adaptive. Adaptive will handle most tasks well and keep costs down. But for situations where you know you need careful reasoning — for instance, when you are working across different repositories and making a stringent plan that requires substantial thinking — that is when you should reach for something like Claude Code or another premium model. Match the model to the task.

Pausing the Demo

Dileep then announced he would pause the Windsurf walkthrough here. He wanted to do the actual Superplexity planning and the live build on Day 2, partly because new people would likely join tomorrow and they should be able to see the planning phase too. So Day 2 would pick up from this exact point: plan out Superplexity, then build it, and then also show what the experience is like on Devin specifically.

He also flagged that Day 2 would cover the **work tree** concept he had deferred today.

Introducing the Outskill Illumini Forge

Handing Off to Shashwat

Dileep then introduced Shashwat, a new addition to the Outskill team who had conceptualised and worked independently on a new initiative. He framed the introduction warmly — appreciating Shashwat publicly for his work, and asking the audience to be kind because young people early in their roles can occasionally trip up. The audience welcomed him with messages of encouragement.

Shashwat took over and signposted that the announcement would take ten to fifteen minutes. He started with a pulse check — three questions:

How many people had been stuck on a bug or a prompt recently and just given up out of frustration? How many had shipped a script, mini-project, or full product and then felt that nobody who really understood the AI space had seen it? How many had highly specific AI questions but did not know where to ask them?

The chat showed that almost everyone had experienced at least one of the three, and many had experienced all three. This was the gap the new initiative was designed to fill.

What the Forge Is

The new initiative is called the **Outskill Illumini Forge**. Shashwat described it as not just a meetup, not just a collaboration space, but a mix of all of them — collaboration, networking, meetups, product showcases, and more. The framing was that learning does not end when the course ends. Building does not happen alone. The Forge is the space for what comes next.

He emphasised three points about the Forge:

It is a space built by the community and led by the community. The community here is the alumni — the people who have come through the Outskill cohorts. The best part — it is free. No extra cost, no hidden charges. It is a place where people who have already gone through the cohorts hang out, ship together, make products, discuss products, and pull each other up.

The Five Needs the Forge Addresses

Rather than guess at what alumni wanted, the team had mapped out the specific needs that had emerged from earlier feedback. Shashwat walked through them:

People needed help when stuck — somewhere to bring blockers and get unstuck. People wanted a place to show what they had built — for visibility and feedback. People wanted other builders to talk to about ideas and builds. People wanted AI updates that actually mattered — curated, relevant, not noise. People wanted to connect with co-founders, gigs, jobs, and freelance opportunities in the AI space.

For each of these needs, the Forge has a dedicated space.

The Five Spaces

The five spaces map directly to the five needs:

Peer Support	The space for getting help when stuck. Across all programs — AIAP, AI Generalist, Catalyst, and others — peers and mentors will be present. If you hit a bug, a blocker, or a confusing idea, drop a screenshot or a description into Peer Support. Chances are someone else has hit the same problem or is hitting it right now, and you can solve it together.
Wins & Showcases	Shashwat's personal favourite, which he called the "hype room." This is where shipping becomes a habit and milestones get celebrated. Whether you have shipped a full MVP, or you have just started working on a new idea on a Sunday afternoon, post it. When you share your win, you also inspire the people who are still hesitating to start.
General Chat	A virtual water cooler. Drop a random thought, share a weird tool you found, ask any stupid AI-related question, roast an AI launch. Some of the best co-founder pairings, he noted, just start with a random message.
AI Pulse	A news filter for the AI world. Found a new model drop? Drop it on AI Pulse. An API change? AI Pulse. The channel becomes a curated feed of what is happening in the AI space, contributed by everyone.
Opportunity Hub	The networking and jobs space. Full-time roles posted by builders for builders. Freelancer postings. Co-founder matching when someone is ready to build something serious.

There is also a Google Form link pinned in the Wins and Showcases space — “Ready to demo your product live.” You submit your product through the form, the team reviews it, and if it is a good fit they reach out to discuss featuring it in a live product showcase.

Where to Find the Forge

Shashwat then shared his screen to walk through the LMS — the platform.outskill.com space. The Outskill Illumini Forge would show up there with all five spaces, plus an Announcement space at the top.

The Announcement space is admin-controlled. Anything regarding sessions, offline meetups, networking events, or the Forge itself would be posted there to keep everyone updated.

He noted that the spaces would go live the following day. Some people were already asking why they could not see the Forge on their LMS — the answer was that the spaces would become available from midnight onward, so they should wait until the next day to start using them.

The First Live Product Showcase

First live product showcase · Friday, 15th May 2026 · 7:30 PM IST · Two community products selected for the inaugural event.

The format would be a live teardown — the founders would showcase their products, explain how they had built them, walk through the technical and design decisions, and discuss how they were scaling. Both products are built by Outskill alumni. After the showcase, there would be space for the audience to network with the founders.

For anyone who wanted a slot in a future showcase, the Google Form link in the Wins and Showcases space was the way to apply.

Shashwat's Closing Ask

Shashwat closed his portion with a simple ask. When the LMS space went live the next day, the first move for each participant should be:

Log in. Drop one sentence in the General Chat about who you are and what you are building. Go to Peer Support and either ask a question about a broken workflow or share something you have shipped, even if it is half-broken or not fully scalable. Network with peers in the Forge.

He thanked the audience and handed back to Kartik.

Community Guidelines for the Forge

Kartik's Wrap-Up

Kartik took back the call to close the session. He confirmed the audience's enthusiasm — many had typed "Forge" in the chat as he had asked. He then walked through a set of community guidelines that participants would need to follow once the Forge went live.

The Nine Guidelines

01 Show up as a builder, not a spectator.

Lurking is fine for the first week as you orient yourself. After that, contribution is expected — and not in a token way. It is a responsibility shared by everyone in the community. The Forge works because people show up and contribute, not because they passively consume.

02 Do not sell or spam.

If you have something to pitch, an opportunity to offer, or an opportunity you are looking for, the Opportunity Hub is the dedicated space for it. Do not slide into people's DMs. Do not push promotional content into other channels. Respect the boundaries of each space.

03 Search before you ask.

Outskill is now a community of more than 10,000 people. Before asking a question, check whether someone has already asked it. There is a high probability that the answer is already out there. This respects everyone's time and keeps the channels productive.

04 Credit people always.

When somebody supports you — debugs your code, suggests an idea, helps you unblock — give them credit publicly. Kartik noted he would feel happy if Shashwat gave him credit for support, and that everyone in the community works the same way. Reputation in the Forge is built through public acknowledgement of the help you receive.

05 What is said in the Forge stays in the Forge.

Members will share work in progress, revenue numbers, struggles, and personal stories. Treat the Forge like a trusted room. Do not share content from the Forge outside of it. The trust that makes vulnerable sharing possible depends on this norm being upheld.

06 Keep it to the LMS — no WhatsApp.

Outskill does not endorse or entertain unofficial WhatsApp groups. Please do not create them. Conversations should happen on the LMS where the team can support the community properly and where the institutional knowledge accumulates.

07 Respect, respect, respect.

Be respectful to peers. Be respectful to mentors. Be respectful to people who come in to demo their work, to ask questions, to seek opportunities. Disrespect of any kind is not tolerated.

08 Sessions are a goldmine — missing one is your loss.

Weekly AI updates, AI sprints, and now the product showcases are all part of the regular rhythm. Attending live is consistently more valuable than catching up on the recording. Make a point of joining live whenever you can.

09 Email IDs are identifiers.

Do not use multiple email IDs to join live sessions or to access the LMS. Stick to one email ID — two at most. The team uses email IDs as the canonical identifier for participants, and proliferating IDs makes it harder to track participation, deliver certificates, and provide proper support.

Final Encouragement

Kartik closed with a single line: “Let’s collaborate and build something messy together.” He asked the audience to type “build” in the chat if they were on board. The chat lit up with “build” messages.

He confirmed the schedule. Day 2 of the AI Sprint would happen tomorrow. The first live product showcase would happen on Friday. The cadence of these initiatives going forward would be roughly

every two weeks, every three weeks, every month — at least once a month minimum. The team was committed to bringing more and more value to the community on a regular basis, even if the specific frequency would vary.

Closing Notes on Logistics

Kartik covered the standard logistics:

Recordings of the session would be available by the next morning, Indian Standard Time, before 11 AM. This was the standard turnaround time. He addressed a question from someone called Shams about wanting recordings faster — the bottleneck is Zoom, which takes four to five hours to process the recording before it can be uploaded. The team uploads it as soon as Zoom releases it.

Post-read materials would also be available, following the usual process. Anyone who attended the session live would, after Day 2, receive a certificate for completing the AI Sprint on Windsurf and Devin. The certificate would be issued the day after the Day 2 session.

He encouraged participants to inform peers about the AI Sprints and Masterclasses so the community could grow.

He thanked everyone for joining, said he looked forward to seeing them on Day 2, and closed the session.

Themes and Takeaways

The Big Idea of the Session

If the session had one big idea, it is that producing reliable software with AI requires moving from conversation to specification. Vibe coding is a conversation — informal, exploratory, forgiving. Agentic coding is a specification — formal, precise, demanding. The transition between the two is not just about using a different tool; it is about thinking differently about what you are asking the AI to do and what you owe it in the way of clarity.

The Engineering Mindset

Dileep was teaching what he called Engineering 101. Not the syntax of any particular language. Not the features of any particular tool. The discipline of thinking about how things go wrong before they go wrong. The four concepts — edge cases, authentication and authorization, rate limits, and error handling — are the basic mental checklist that separates a hobbyist build from a product. If you can answer the question “how could this break?” for each of these four categories, you are most of the way to a robust application.

Why Analogies Carried the Session

The session was unusual in how much it relied on analogies — the pizza restaurant for GitHub, the grandma’s recipe versus MasterChef for conversation versus specification, the cinema for authentication versus authorization, the dinner guests for rate limits and error handling. Dileep was explicit about why he chose this approach. The analogies are not perfect, but they make the concepts portable. You can now explain GitHub to a non-technical friend. You can explain why your app needs error handling without using the word “exception.” You can explain authentication without making your audience’s eyes glaze over.

The audience for these sessions is mixed — engineers and non-engineers, technical and non-technical, people who have used GitHub for years and people who have never opened it. The analogies are the lowest common denominator that makes the conceptual content land for everyone.

The Shift in How Documentation Is Written

A small but telling point Dileep made was that API documentation has become agent-friendly rather than human-friendly. Documentation sites now expose themselves as markdown for LLMs, with buttons to

“Open in Claude” or “Connect to Cursor.” This is a quiet but significant shift in how software ecosystems are being built — assuming that the primary reader is going to be an AI agent, not a human developer. If you are doing agentic coding, you are working with the grain of this shift.

The Path Through the Two Days

The session set up a clear path. Day 1 was foundations — the why and the what. The transition from vibe coding to agentic coding, the GitHub primer, the specification mindset, the four engineering concepts, the lineage of the IDEs, a brief tour of Windsurf. Day 2 would be the application of all this — actually planning Superplexity, actually building it in Windsurf 2.0 with Devin, demonstrating what work trees are, and showing what it looks like to do agentic coding on a real codebase rather than a toy example.

The Community Layer

The session also marked the introduction of the Outskill Illumini Forge — the community layer that lives around the courses. Five spaces, a clear set of guidelines, a first live product showcase scheduled for Friday, and a commitment to ongoing regular content. The Forge is positioned as the bridge between being a learner and being a builder, with the explicit acknowledgement that the gap between those two states is where most people fall off, and where peer community can do the most good.

The session closed with the audience invited to do exactly what the session had been about — to stop being passive learners and start being active builders, supported by a community that would help them when they got stuck and celebrate with them when they shipped.